

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Juhi Surin	Reg. No.:	192525148
Date:	18-08-2026	Duration:	60 minutes

Code of Conduct

I Juhi Surin (192525148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Juhi Surin

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CI/CD PIPELINE DESIGN REPORT

CO4 Assessment Tool 2 – Software Engineering (CSA10)

Assigned Problem Statement (Q37):

“AI-Based Digital Waste Recycling Marketplace and Resource Recovery System”

Submitted by:	Juhi Surin
Registration Number:	192525148
Course:	CSA1013 – Software Engineering
Faculty In-Charge:	Dr. R. Latha
Department:	B.Tech AIML
Date of Submission:	18 August 2026

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 — CI/CD Pipeline Design Exercise

Problem 37: AI-Based Digital Waste Recycling Marketplace and Resource Recovery System

1. Problem Statement Summary

Existing waste management systems lack effective coordination between waste generators, recycling facilities, and buyers of recyclable materials, resulting in poor recycling rates and increased landfill waste. The proposed AI-powered digital marketplace connects households, industries, recyclers, and waste collection agencies; automatically classifies recyclable materials using AI; optimizes collection schedules; recommends nearby recycling centers; facilitates trading of recyclable materials; and generates sustainability analytics to improve resource recovery.

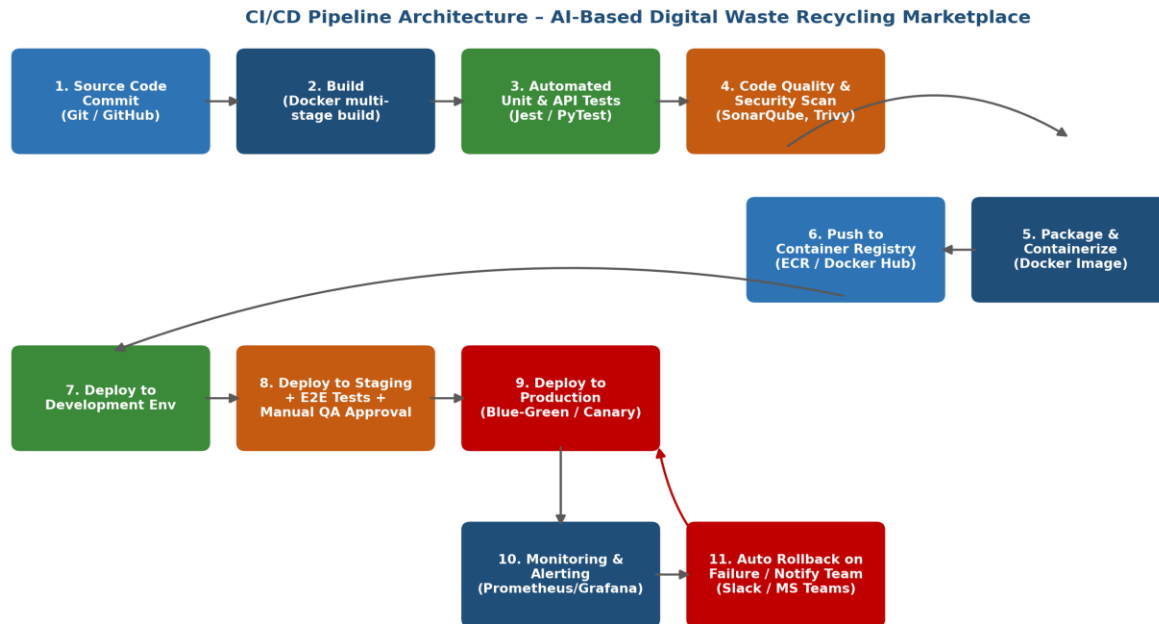
2. Requirement & CI/CD Needs Analysis

The system consists of three cooperating components — a React web/mobile frontend, a Node.js marketplace/scheduling API, and a Python-based AI waste-classification service — that must be integrated, tested, and released frequently without disrupting live trading or collection operations. The table below maps each project requirement to the corresponding CI/CD need.

Project Requirement / Risk	Resulting CI/CD Need
Frequent code changes across 3 sub-systems (React frontend, Node.js marketplace API, Python AI classification service) must integrate without breaking existing functionality.	Continuous Integration with automated build + unit tests on every commit.
The AI waste-classification model and APIs must be validated for accuracy and response time before reaching users.	Automated model/API testing stage with defined pass thresholds (e.g., accuracy $\geq 85\%$, latency $\leq 3s$).
Multiple stakeholder roles (household, industry, recycler, agency) mean any bug can disrupt trading, scheduling, or analytics for many users at once.	Staging environment with full regression + manual QA approval gate before production deploy.
The platform must stay available and trustworthy (financial/trade transactions, personal data).	Security scanning (SAST/DAST, container scan) and a fast, automated rollback mechanism.
Growing user base and seasonal spikes in waste-collection requests require elastic scaling.	Containerized microservices deployed on an orchestrator (Kubernetes) with auto-scaling.

3. CI/CD Pipeline Architecture & Workflow

The pipeline below takes code from commit to production through eleven automated/gated stages, ensuring every change is built, tested, quality-checked, security-scanned, and safely promoted through Development → Staging → Production, with continuous monitoring and automatic rollback.



Workflow narrative (commit to deployment):

- A developer commits code to a feature branch and opens a pull request to GitHub; this triggers the GitHub Actions pipeline automatically.
- The Build stage compiles/bundles the frontend, backend, and AI service and produces Docker images using multi-stage builds.
- Automated unit and API tests run against each service; failures stop the pipeline immediately and notify the developer.
- Code Quality and Security scans (SonarQube, Trivy) enforce a quality gate — critical issues block the pipeline before any artifact is published.
- Passing builds are packaged into versioned Docker images and pushed to the container registry.
- The pipeline auto-deploys to the Development environment for early integration feedback.
- On merge to main, the build is promoted to Staging, where E2E and performance tests run automatically, followed by a manual QA/UAT approval gate.
- Once approved, a tagged release is deployed to Production using a Blue-Green/Canary strategy to minimize risk to live users.
- Prometheus/Grafana continuously monitor error rates, latency, and classification throughput; if thresholds are breached, the pipeline automatically rolls back to the last stable version and notifies the team via Slack/MS Teams.

4. Tools & Technology Selection

Each pipeline stage uses a tool selected for its fit with the system's tech stack, automation support, and industry adoption, as justified below.

Pipeline Stage	Tool(s) / Technology	Justification
Source Code Management	Git + GitHub	Industry-standard distributed VCS; supports branch protection, pull-request reviews, and native webhook triggers for CI.
CI Orchestration	GitHub Actions	Tightly integrated with the GitHub repo, YAML-based pipelines, free tier suffices for an academic/startup-scale project, large marketplace of pre-built actions.
Build	Docker (multi-stage builds) + npm/Node build, pip/Python build	Produces small, reproducible, environment-independent images for the React frontend, Node.js API, and Python AI service.
Unit / API Testing	Jest (Node.js & React), PyTest (Python AI service)	Fast, widely-adopted frameworks matching each service's language; generate coverage reports consumable by the CI dashboard.
Code Quality Analysis	SonarQube + ESLint / Pylint	Detects code smells, duplication, and complexity; enforces a quality gate that blocks the pipeline if standards aren't met.
Security Scanning	OWASP Dependency-Check / Snyk (dependencies), Trivy (container images)	Identifies known CVEs in libraries and base images before they reach production.
Artifact / Package Registry	Docker Hub / Amazon ECR	Central, versioned storage for container images with access control and vulnerability scanning.
Deployment / Orchestration	Kubernetes (Amazon EKS) + Helm charts	Enables declarative, scalable deployment of Dev/Staging/Prod environments and supports blue-green/canary rollouts.
Infrastructure as Code	Terraform	Version-controlled, repeatable provisioning of cloud infrastructure (clusters, databases, networking).
Monitoring & Alerting	Prometheus + Grafana	Real-time metrics on API latency, AI classification throughput, and error rate, feeding the rollback decision.
Notifications	Slack / MS Teams Webhooks	Immediate pipeline success/failure alerts to the development team for fast response.
Secrets Management	GitHub Secrets / HashiCorp Vault	Keeps API keys, database credentials, and cloud tokens out of source code.

5. Automated Testing Strategy

Testing is layered so that cheap, fast checks (unit tests) run first and expensive checks (performance, E2E) run later, giving developers rapid feedback while still guaranteeing release quality.

Test Type	When It Runs	Tool(s)	Purpose / Pass Criteria
Unit Testing	Every commit / PR	Jest, PyTest	Validates individual functions (e.g., quantity validation, price calculation) in isolation; must pass with $\geq 80\%$ coverage.
API / Integration Testing	After successful build	Supertest (Node), Postman/Newman	Confirms marketplace, scheduling, and classification endpoints return correct responses and handle errors gracefully.
AI Model Validation	After build, before packaging	Custom PyTest suite with labeled sample dataset	Confirms the waste-classification model meets a minimum accuracy threshold ($\geq 85\%$) and response time ($\leq 3s$) before deployment.
Static Code Analysis	In parallel with testing	SonarQube, ESLint, Pylint	Flags code smells, duplicated logic and complexity issues; pipeline fails the quality gate on critical issues.
Security Testing (SAST/DAST + Container Scan)	Before packaging & before staging deploy	Snyk, OWASP ZAP, Trivy	Detects vulnerable dependencies, insecure code patterns, and image CVEs.
End-to-End (E2E) Testing	In Staging environment	Cypress / Selenium	Simulates full user journeys — listing waste, AI classification, trade completion, schedule request — across all roles.
Performance Load Testing	Before production release	Apache JMeter	Confirms the platform handles concurrent load (e.g., 500 concurrent users) within acceptable response times.
User Acceptance Testing (UAT)	Manual gate in Staging	Manual QA sign-off	Final human check of new features against requirements before promotion to production.

6. Deployment Strategy — Environments

Environment	Trigger	Configuration	Purpose
Development	Auto-deployed on every merge to the `develop` branch	Single replica, mock/sandbox data, relaxed feature flags	Fast feedback for developers; not customer-facing.
Staging Testing	Auto-deployed on merge to `main`, gated by passing tests + quality gate	Mirrors production configuration and scale (smaller capacity), anonymized production-like data	Full regression, E2E, performance testing and manual QA/UAT approval before promotion.
Production	Deployed on tagged release, only after Staging sign-off	Multi-replica, auto-scaling, full monitoring and alerting enabled	Blue-Green or Canary strategy limits exposure of new code; live user traffic.

Production releases use a Blue-Green deployment: the new version ('green') is deployed alongside the current stable version ('blue') and receives traffic only after automated health checks pass; for higher-risk releases, a Canary strategy routes a small percentage of live traffic (e.g., 5–10%) to the new version before a full rollout.

7. Security, Quality Checks, Notifications & Rollback Mechanisms

7.1 Security

- Secrets (API keys, DB credentials, cloud tokens) are stored in GitHub Secrets / HashiCorp Vault — never committed to source code.
- Dependency scanning (Snyk/OWASP Dependency-Check) and container image scanning (Trivy) run on every build to catch known CVEs before deployment.
- Role-based access control (RBAC) restricts who can approve production deployments or modify pipeline configuration.
- All inter-service and external traffic uses HTTPS/TLS; database access is restricted to private subnets.

7.2 Quality Checks

- A SonarQube quality gate blocks promotion if code coverage falls below 80% or critical/blocker issues are detected.
- Linting (ESLint/Pylint) enforces consistent code style automatically on every pull request.

7.3 Notifications

- Slack/MS Teams webhooks post real-time build, test, and deployment status (success or failure) to the development channel.
- Failed pipeline runs automatically assign/notify the responsible developer via GitHub PR comments and Slack.

7.4 Rollback Mechanism

- Kubernetes readiness/liveness probes detect unhealthy pods immediately after deployment and stop traffic routing to them.
- If Prometheus alerts cross defined thresholds (e.g., error rate > 5%, latency > 3s) within a post-deploy monitoring window, the pipeline automatically re-routes traffic back to the previous stable ('blue') version.
- Every deployment is tagged and stored in the container registry, allowing a one-command manual rollback to any prior stable release if needed.

8. Conclusion

This CI/CD pipeline automates the journey from code commit to production for the AI-Based Digital Waste Recycling Marketplace, integrating automated testing, code quality and security gates, staged environments, and monitoring-driven rollback. This ensures frequent, reliable releases while protecting the platform's availability, data security, and trustworthiness for all stakeholders — households, industries, recyclers, and waste collection agencies.